Vietnam National University - HCMC University of Science

Faculty of Information Technology

---□📖□---

Software Testing

Course: Introduction to Software Engineering

Students that participated in this project:

Đinh Ngọc Anh Dương – 23127039

Trần Thị Xuân Tài – 23127256

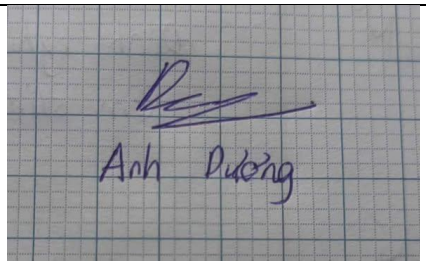
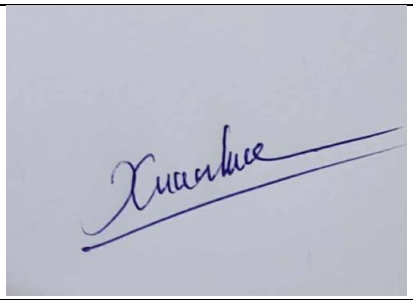
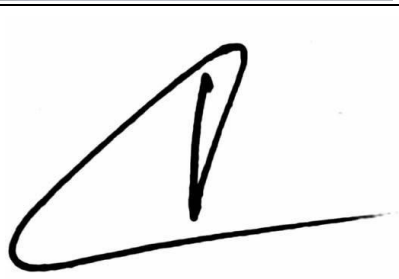
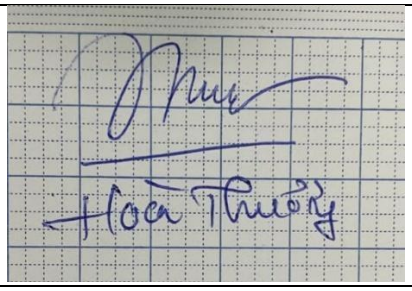
Trần Thọ - 23127264

Nguyễn Tuấn An – 23127316

Đỗ Thị Hoài Thương - 23127492

Instructor: Trương Phước Lộc

I. Member Contribution Assessment

ID	Name	Contribution(%)	Signature
23127039	Đinh Ngọc Anh Dương	100%	
23127256	Trần Thị Xuân Tài	100%	
23127264	Trần Thọ	100%	
23127316	Nguyễn Tuấn An	100%	
23127492	Đỗ Thị Hoài Thương	100%	

II. Test Plan

1. Testing Objectives

The objective of this test plan is to ensure that the Unicode Programming Practice System operates correctly, reliably, and securely according to its specified functional and non-functional requirements. The testing activities aim to verify the correctness of core functionalities, validate system integration between frontend and backend, and ensure that security mechanisms such as authentication and cross-origin communication are properly enforced.

2. Testing Scope

The scope of testing covers both the backend and frontend components of the system. On the backend side, testing focuses on RESTful APIs, business logic implemented in service layers, database interactions, and security mechanisms including JWT-based authentication and authorization. On the frontend side, testing targets user interface components, form validation, data rendering, and interaction with backend APIs. Configuration-related aspects such as CORS policies are also included in the testing scope. Performance and stress testing at large scale are considered outside the scope of this plan.

3. Testing Strategy and Techniques

The testing strategy adopts a layered approach in order to achieve comprehensive coverage of the system.

At the unit level, individual backend methods are tested in isolation to verify their internal logic and error handling. This includes validation of user input, authentication logic, and code execution handling. White-box testing techniques are applied at this level to ensure that different execution paths within each method are exercised.

Integration testing is performed to validate the interaction between multiple backend components, such as controllers, services, repositories, and security filters. This level of testing ensures that data flows correctly through the system, that authentication tokens are properly generated and validated, and that API endpoints behave correctly when interacting with the database and execution services.

System testing is conducted to evaluate the complete application as an integrated whole. This level verifies end-to-end workflows from a user perspective, such as registering an account, logging in, browsing problems, submitting code, and receiving evaluation results. The goal is to ensure that all system components work together as expected in a realistic usage scenario.

Frontend functional testing focuses on validating user interface behavior and user interactions. This includes verifying form input validation, page navigation, dynamic rendering of problem lists and details, and interaction with backend APIs. Black-box testing techniques are primarily used to assess whether the frontend behaves correctly based on user actions and received responses.

Security and configuration testing is applied to ensure that the system enforces proper access control and secure communication. Authentication and authorization mechanisms are tested to confirm that protected resources cannot be accessed without valid credentials. Cross-origin resource sharing (CORS) configuration is also validated to ensure that frontend and backend communication functions correctly while preventing unauthorized origins.

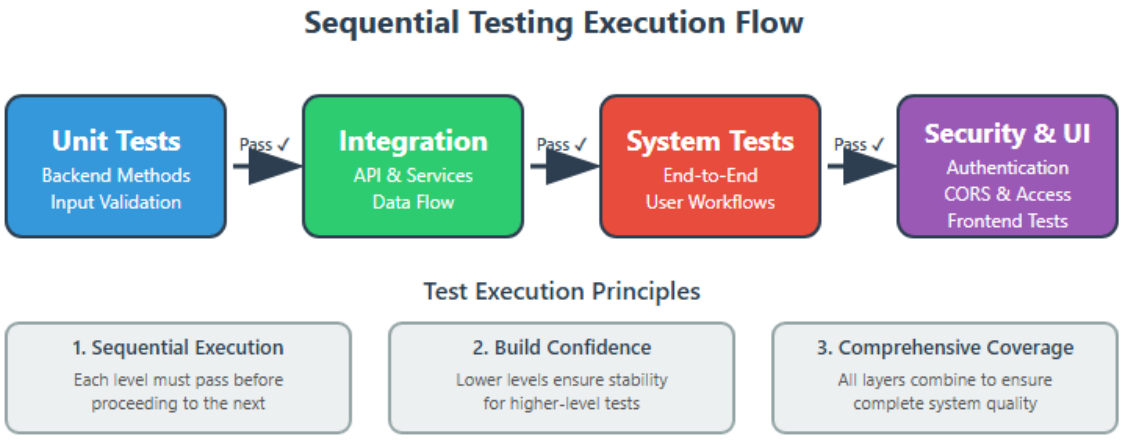


Figure 1: Sequential Testing Execution Flow

4.Test Objects

Testing activities are performed on various system objects, including backend controllers and services responsible for authentication, problem management, submission handling, and code execution. Security-related components such as JWT filters and configuration classes are also tested. On the frontend, core React components related to user authentication, problem listing, and code submission are included as test objects. In addition, configuration files that affect runtime behavior and security are considered part of the testing targets.

5. Test Environment

Testing is conducted in a local development environment that closely mirrors the intended deployment setup. The backend application runs on a Spring Boot server, while the frontend is served using a modern JavaScript development server. The database is provided by PostgreSQL with schema and sample data managed through Flyway migrations. API testing tools and browser developer tools are used to support testing activities and result verification.

III. Test cases

1. List of test cases:

Seq	Test Case	Target	Description
1	User Registration Valid Data	AuthController.register()	Test successful user registration with valid username, email, password
2	User Registration Duplicate Username	AuthController.register()	Test registration failure when username already exists
3	User Login Valid Credentials	AuthController.login()	Test successful login with correct username/password
4	User Login Invalid Credentials	AuthController.login()	Test login failure with incorrect password
5	JWT Token Validation	JwtAuthenticationFilter	Test JWT token validation for protected endpoints
6	Get Problems List with Pagination	ProblemController.getProblems()	Test problem listing with page/size parameters
7	Get Problems with Difficulty Filter	ProblemController.getProblems()	Test filtering problems by difficulty level
8	Get Problem Detail by Slug	ProblemController.getProblemDetailBySlug()	Test retrieving problem details for coding interface
9	Submit Code Valid Solution	SubmissionController.submit()	Test code submission with correct solution

10	Submit Code Invalid Solution	SubmissionController.submit()	Test code submission with wrong answer
11	Submit Code Compilation Error	CodeExecutionService	Test handling of code that fails to compile
12	Submit Code Timeout	JudgeService	Test code execution timeout handling
13	Frontend Problem List Display	ListExercise Component	Test problem list rendering and pagination controls
14	Frontend Code Editor Functionality	ProblemDetail Component	Test code editor, language selection, and submission
15	CORS Configuration	SecurityConfig	Test cross-origin requests from frontend to backend

2. Test case specifications:

2.1. User Registration Valid Data

Test case	User Registration Valid Data
Related Use case	User Registration
Context	User is on the registration page. The backend service is running and database is accessible.
Input Data	Username: "testuser123" Email: "testuser123@gmail.com" Password: "12345678"
Expected Output	- The system accepts the registration - A success message is displayed: "Registration successful!" - User is redirected to login page or dashboard - User record is created in database
Test steps	1. Open browser and navigate to http://localhost:5173/signup 2. Enter username: "testuser123" 3. Enter email: "testuser123@gmail.com" 4. Enter password: "12345678" 5. Click "Sign Up" button 6. Observe the response message 7. Verify in database: SELECT * FROM users WHERE username='testuser123'
Actual Output	- The system accepts the registration - A success message is displayed: "Registration successful!" - User is redirected to login page or dashboard - User record is created in database
Result	Passed

2.2. User Registration Duplicate Username

Test case	User Registration Duplicate Username
Related Use case	User Registration

Context	User "existinguser" already exists in the database. User is attempting to register with the same username.
Input Data	Username: "testuser123" (already exists in DB) Email: "newemail@gmail.com" Password: "12345678"
Expected Output	- Registration is rejected - Error message: "Username đã tồn tại" - User stays on registration page - No duplicate record in database
Test steps	1. Create existing user first (use Test Case 1) 2. Navigate to http://localhost:5173/signup 3. Enter username: "testuser123" (same as before) 4. Enter email: "newemail@gmail.com" 5. Enter password: "12345678" 6. Click "Sign Up" button 7. Observe error message 8. Verify database count: SELECT COUNT(*) FROM users WHERE username='testuser' (should be 1, not 2)
Actual Output	- Registration is rejected - Error message: "Username đã tồn tại" - User stays on registration page - No duplicate record in database
Result	Passed

2.3. User Login Valid Credentials

Test case	**User Login Valid Credentials**
Related Use case	User Authentication
Context	User "testuser123" is registered in the system with password "12345678". User is on the login page.
Input Data	Username: "testuser123" Password: "12345678"
Expected Output	- Login successful - JWT token is returned - User is redirected to problems page

	- Token is stored in localStorage
Test steps	1. Ensure user exists (create via Test Case 1 if needed) 2. Navigate to http://localhost:5173/login 3. Enter username: "testuser123" 4. Enter password: "12345678" 5. Click "Login" button 6. Observe response and redirection 7. Open DevTools > Application > Local Storage 8. Verify "jwt_token" exists
Actual Output	- Login successful - JWT token is returned - User is redirected to problems page - Token is stored in localStorage
Result	Passed

2.4. User Login Valid Credentials

Test case	**User Login Invalid Credentials**
Related Use case	User Authentication
Context	User "testuser123" exists with correct password "12345678". User attempts to login with wrong password.
Input Data	Username: "testuser123" Password: "87654321" (incorrect)
Expected Output	- Login fails - Error message: "Sai password" - HTTP 401 Unauthorized status - User stays on login page - No token issued
Test steps	1. Navigate to http://localhost:5173/login 2. Enter username: "testuser123" 3. Enter password: "87654321" 4. Click "Login" button 5. Observe error message 6. Verify no token in localStorage 7. Verify users are not redirected
Actual Output	- Login fails

	- Error message: "Sai password" - HTTP 401 Unauthorized status - User stays on login page - No token issued
Result	Passed

2.5. JWT Token Validation

Test case	**JWT Token Validation**
Related Use case	JWT Token
Context	The system uses JWT-based authentication, protected endpoints require a valid JWT token, JwtAuthenticationFilter validates the token before allowing access
Input Data	API endpoint (protected): /api/problems HTTP Header: • Authorization: Bearer <JWT_TOKEN>
Expected Output	**Valid Token** • Request is processed successfully • API returns **200 OK** **Missing Token** • Request is rejected • API returns **401 Unauthorized** **Invalid or Expired Token** • Request is rejected • API returns **401 Unauthorized**
Test Steps	1.Send a request to /api/problems **without** JWT token 2. Verify that the response status is **401 Unauthorized**

	3. Send a request with an **invalid JWT token** 4. Verify that the response status is **401 Unauthorized** 5. Send a request with a **valid JWT token** 6. Verify that the response status is **200 OK**
Actual Output	Requests without token are rejected Requests with invalid or expired token are rejected Requests with valid token are processed successfully
Result	Passed

2.6. Get Problems List with Pagination

Test case	**Get Problems List with Pagination**
Related Use case	Get Problems List
Context	The user navigates to the problem list page The system retrieves problems from the backend using pagination parameters The frontend displays the problem list based on the current page and page size
Input Data	URL example: http://localhost:5173/problems API endpoint: /api/problems API request parameters: • page = 0 • size = 10 • difficulty = null • tags = null • search = null
Expected Output	**Loading State**: While data is being fetched, the

	message **“Loading problems…”** is displayed **Problem List Rendering**: The system displays a maximum of **10 problems** on the first page • Each problem shows basic information (title, difficulty, tags) **Pagination Rendering:** • Pagination controls (Next / Previous or page numbers) are displayed • User can navigate to the next page to load additional problems
Test Steps	1. Open the URL /problems 2. Verify that the **“Loading problems…”** message is displayed initially 3. Wait until the data finishes loading 4. Verify that **10 problems** are displayed 5. Navigate to the next page using pagination controls 6. Verify that a new set of problems is loaded
Actual Output	The problem list page loads successfully The loading message appears while fetching data Problems are displayed according to the page size Pagination works correctly
Result	Passed

2.7. Get Problems with Difficulty Filter

Test case	**Get Problems with Difficulty Filter**
Related Use case	Get Problems

Context	The user applies a difficulty filter The system retrieves problems matching the selected difficulty from the backend The frontend displays only problems with the selected difficulty
Input Data	URL example: http://localhost:5173/problems API endpoint: /api/problems API request parameters: • page = 0 • size = 10 • difficulty = "EASY" • tags = null • search = null
Expected Output	**Loading State** • The message **"Loading problems…"** is displayed while fetching data **Filtered Problem List Rendering** • All displayed problems have difficulty **EASY** • No MEDIUM or DIFFICULT problems are shown **Empty State** • If no problems match the filter, display: **"No problems found"**
Test Steps	1. Open the URL /problems 2. Select the **EASY** difficulty filter 3. Verify that the loading message is displayed 4. Wait until the data finishes loading

	5. Verify that all displayed problems have difficulty **EASY** 6. Verify the fallback message if no problems exist
Actual Output	The loading message appears correctly The system fetches problems with difficulty **EASY** Only EASY problems are displayed Appropriate fallback message is shown when needed
Result	Passed

2.8 Get Problem Detail by Slug

Test case	**Get Problem Detail by Slug**
Related Use case	Get Problem Detail
Context	The user selects a problem from the problem list The system retrieves problem details using the problem slug The frontend displays full problem information
Input Data	URL example: http://localhost:5173/problems/two-sum API endpoint: /api/problems/two-sum API request parameter: slug = "two-sum"
Expected Output	**Loading State** • The message **"Loading problem detail…"** is displayed while fetching data **Problem Detail Rendering** • The problem title is displayed correctly • The difficulty level is shown • The full problem description is rendered

	• Examples and constraints are displayed (if available) **Error Handling** • If the slug does not exist, display: ○ **"Problem not found"**
Test Steps	1. Open the URL /problems/two-sum 2. Verify that the loading message is displayed 3. Wait until the data finishes loading 4. Verify that the problem title and difficulty are correct 5. Verify that the problem description is displayed 6. Test with an invalid slug and verify the error message
Actual Output	The problem detail page loads successfully The loading message appears while fetching data Problem details are rendered correctly Error handling works as expected
Result	Passed

2.9. Submit Code Valid Solution

Test case	Submit Code Valid Solution
Related Use case	Submit Solution / Solve Problem
Context	User is logged in and is currently viewing the "Problem Detail" page for a specific problem (e.g., "Sum of Two Numbers"). The backend service is running.
Input Data	Problem ID: 1 Language: C++ Source Code: A valid C++ solution that passes all test cases.
Expected Output	The system accepts the submission. The SubmissionController returns a success status.

	The UI displays a "Accepted" or "Passed" status. The user's solved problem count increments.
Test steps	1. Log in to the application with valid credentials. 2. Navigate to the problem page for Problem ID 1. 3. Select "C++" from the language dropdown. 4. Paste the valid source code into the code editor. 5. Click the "Submit" button. 6. Observe the result displayed on the screen.
Actual Output	ACCEPTED
Result	Passed

2.10. Submit Code Invalid Solution

Test case	**Submit Code Invalid Solution**
Related Use case	Submit Solution / Solve Problem
Context	User is logged in and viewing the "Problem Detail" page.
Input Data	Problem ID: 1 Language: Python Source Code: A syntactically correct code that produces incorrect output (logic error).
Expected Output	The system processes the submission. The JudgeService detects the output mismatch. The UI displays a "Wrong Answer" status. The specific test case that failed is indicated (if applicable).
Test steps	1. Log in to the application. 2. Navigate to the problem page. 3. Enter code that compiles but fails the logic test (e.g., returns sum + 1 instead of sum). 4. Click the "Submit" button. 5. Observe the result displayed on the screen.
Actual Output	WRONG_ANSWER (No errror details)
Result	Passed

2.11. Submit Code Compilation Error

Test case	Submit Code Compilation Error
Related Use case	Submit Solution / Solve Problem
Context	User is logged in and viewing the "Problem Detail" page.
Input Data	Problem ID: 1 Language: C++ Source Code: Code containing syntax errors (e.g., missing semicolon, undefined variable).
Expected Output	The CodeExecutionService fails to compile the code. The UI displays a "Compilation Error" status. The compiler error message is shown to the user in the console/log area.
Test steps	1. Log in to the application. 2. Navigate to the problem page. 3. Enter code with a deliberate syntax error (e.g., int x = without a value). 4. Click the "Submit" button. 5. Verify that the system reports a compilation error.
Actual Output	COMPILATION_ERROR Error Output: solution.cpp:17:26: error: expected ';' before '}' token
Result	Passed

2.12. Submit Code Timeout

Test case	Submit Code Timeout
Related Use case	Submit Solution / Solve Problem
Context	User is logged in. The problem has a defined time limit (e.g., 1 second).
Input Data	Problem ID: 1 Language: C++ Source Code: Code containing an infinite loop or an algorithm with time complexity exceeding the limit (e.g., O(N^3) where O(N) is required).

Expected Output	The JudgeService interrupts the execution after the time limit is reached. The UI displays a "Time Limit Exceeded" (TLE) status. The system resources are released correctly.
Test steps	1. Log in to the application. 2. Navigate to the problem page. 3. Enter code that includes an infinite while(true) loop. 4. Click the "Submit" button. 5. Verify that the system returns a "Time Limit Exceeded" error rather than hanging indefinitely.
Actual Output	TIME_LIMIT_EXCEEDED
Result	Passed

2.13. Frontend Problem List Display

Test case	**Frontend Problem List Display**
Related Use case	View Problem List
Context	The user navigates to a problem list filtered by category The system retrieves problems from the backend with category-based tags. The frontend groups the retrieved problems by difficulty into **Easy** - **Medium** – **Difficult** columns
Input Data	URL example: http://localhost:5173/category/sql Category ID: sql Tag mapping: • sql → database • dsa → algorithm-data-structure • implementation → implementation • debugging → debugging • system-design → system-design • oop → oop API request parameters: • page = 0 • size = 100 • tags = ["database"]
Expected	• **Loading State:** While data is being fetched, the message

Output	**"Loading problems…"** is displayed • **Category Title Rendering:** The page title matches the selected category**.** Example: /category/sql displays **"Database / SQL Coding Questions"** • **Problem List Rendering** - Problems with difficulty = EASY are displayed in the **Easy** column - Problems with difficulty = MEDIUM are displayed in the **Medium** column - Problems with difficulty = DIFFICULT are displayed in the **Difficult** column. • If no problems exist for a difficulty level, the corresponding message is displayed: "No easy problems found" "No medium problems found" "No difficult problems found"
Test Steps	1.Open the URL /category/sql. 2.Verify that the **"Loading problems…"** message is shown initially. 3.Wait until the data finishes loading. 4.Verify that the page title matches the selected category. 5.Verify that: • The **Easy** column displays all problems with difficulty **EASY**, or shows "No easy problems found" if none exist. • The **Medium** column displays all problems with difficulty **MEDIUM**, or shows "No medium problems found" if none exist • The **Difficult** column displays all problems with difficulty **DIFFICULT**, or shows "No medium problems found" if none exist.
Actual Output	• The SpecifiedProblem page loads successfully • The loading message is displayed while fetching data and disappears afterward

	• The category title is displayed correctly • Problems are rendered correctly and grouped into **Easy** – **Medium - Difficult** columns based on their difficulty. • Appropriate fallback messages are displayed when no problems are available for a difficulty level
Result	Passed

2.14. Frontend Code Editor Functionality

Test case	**Frontend Code Editor Functionality**
Related Use case	**Solve Problem in Integrated Code Editor** **View Problem Detail**
Context	• The user is on the Problem Detail page (/problem/:slug). • The language dropdown is rendered inside CodeEditor • Supported languages are **C++**, **Python**, and **JavaScript**. • Changing the selected language triggers reloading of problem details and default code for that language
Input Data	URL: http://localhost:5173/problem/<slug> Language options: • C++ (default) • Python • JavaScript
Expected Output	1. The language dropdown is visible and shows **C++** as the default selection. 2. When a new language is selected: ▪ The system calls getProblemDetail(slug, selectedLanguage.toLowerCase()). ▪ The code editor content is replaced with the **default code of the selected language**. ▪ Any previous run or submission result is cleared. 3. The editor remains editable after the language change. 4. No page reload occurs; the update happens

	dynamically
Test Steps	1. Open the URL /problem/<slug>. 2. Verify that the language dropdown is visible and the default value is **C++**. 3. Observe the default code displayed in the editor. 4. Change the language selection from **C++** to **Python**. 5. Verify that the editor content updates to Python default code. 6. Change the language selection from **Python** to **JavaScript**. 7. Verify that the editor content updates to JavaScript default code. 8. Confirm that any previously displayed run or submission result is no longer shown
Actual Output	• The language selection dropdown is displayed with **C++** selected by default. • When the user selects **Python** or **JavaScript**: - The problem detail API is called with the selected language. - The editor content updates to the corresponding default code. - Previous execution or submission results are cleared. • The editor remains responsive and editable after each language change. • The page updates dynamically without reloading
Result	Passed

2.15 CORS Configuration

Test case	**CORS Configuration**
Related Use case	**Security Configuration**
Context	• Frontend runs on http://localhost:5173 • Backend runs on http://localhost:8080 • CORS is configured to allow all localhost ports via

	http://localhost:*
Input Data	● Origin header: http://localhost:5173 ● Endpoint (public): GET /api/problems ● Endpoint (protected): GET /api/submissions/<id> (requires JWT) ● Method: GET ● Optional preflight: OPTIONS
Expected Output	Requests from http://localhost:5173 are not blocked by the browser Response contains CORS headers, typically: ● **Access-Control-Allow-Origin:** http://localhost:5173 *(or matched origin)* ● **Access-Control-Allow-Credentials**: true ● **Access-Control-Allow-Methods:** GET,POST,PUT,DELETE,OPTIONS Preflight OPTIONS request succeeds (HTTP 200/204) Public endpoints work without JWT; protected endpoints require JWT but **should not fail due to CORS**
Test Steps	1. Start backend server (e.g., localhost:8080). 2. Start frontend on http://localhost:5173. 3. From frontend, trigger a request to GET /api/problems. 4. Open DevTools → Network → inspect response headers. 5. Trigger a request to a protected endpoint GET /api/submissions/<id>: ● without JWT (expect 403/401, but no CORS error) ● with JWT (expect success if token valid) 6. Verify preflight OPTIONS is handled when applicable.
Actual Output	• Requests from http://localhost:5173 reach backend successfully. • No “CORS policy blocked” errors appear in console. • Response headers include CORS allow-origin / allow-credentials values. • Public endpoints succeed; protected endpoints enforce authentication
Result	Passed